Distributed Computing: From Edge to Exascale

Architectures for Intelligent Systems in the Real World

Dr. Mario Harper

Utah State University – DIRECT Lab

Unclassified, Approved for Public Release

ASPIRE
NSF Engineering Research Center

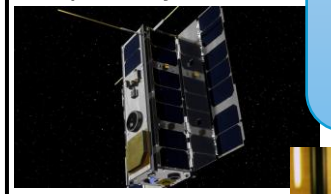
DIRECT LAB

The DIRECT Lab integrates AI, robotics, and electrification to build intelligent, connected infrastructure. We have deployed solutions in space, ground robots, and large-scale transportation systems.

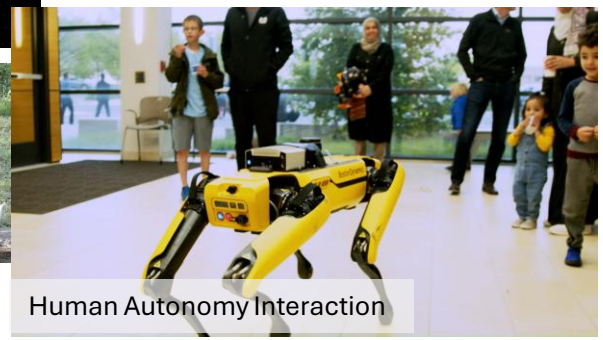
Mario Harper

Autonomy and Robotics

Space Systems

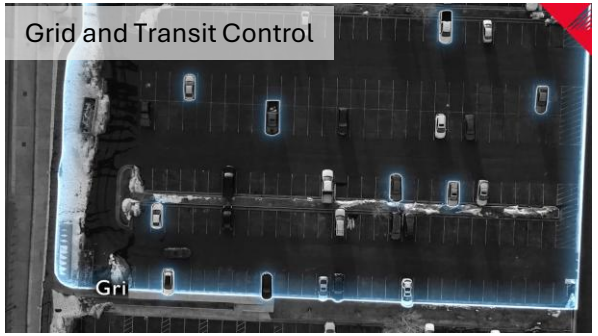


Ground Robots



Human Autonomy Interaction

Electrified Mobility



AI Systems and Digital Twins

Long-Range Mission Simulator

Multiagent Robotics Simulation

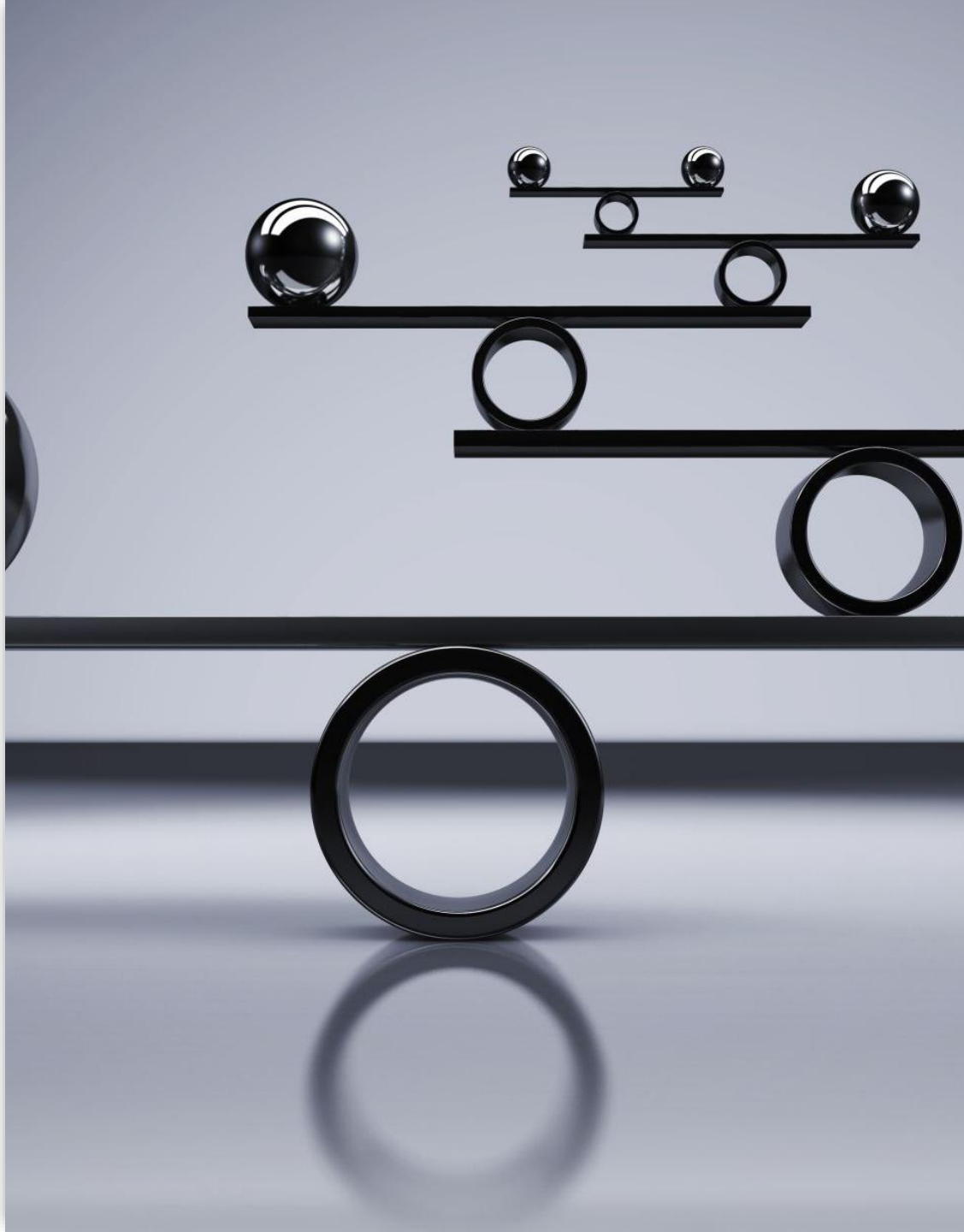
Core Partners

NVIDIA · U.S. ARMY · DEVCOM ARMY RESEARCH LABORATORY · 309th SOFTWARE ENGINEERING GROUP · U.S. AIR FORCE · AlphaZ · NASA · SPACE DYNAMICS LABORATORY

PACIFIC POWER · UTA · GEOTAB · Adobe · ABB · UTAH CLEAN CITIES · Goldman Sachs · NICE · ASI · raisin · driivz

Why Distributed Computing Exists

- Latency constraints
- Physical distribution of sensors
- Data gravity and bandwidth limits
- Fault tolerance and resilience
- Power constraints and scale



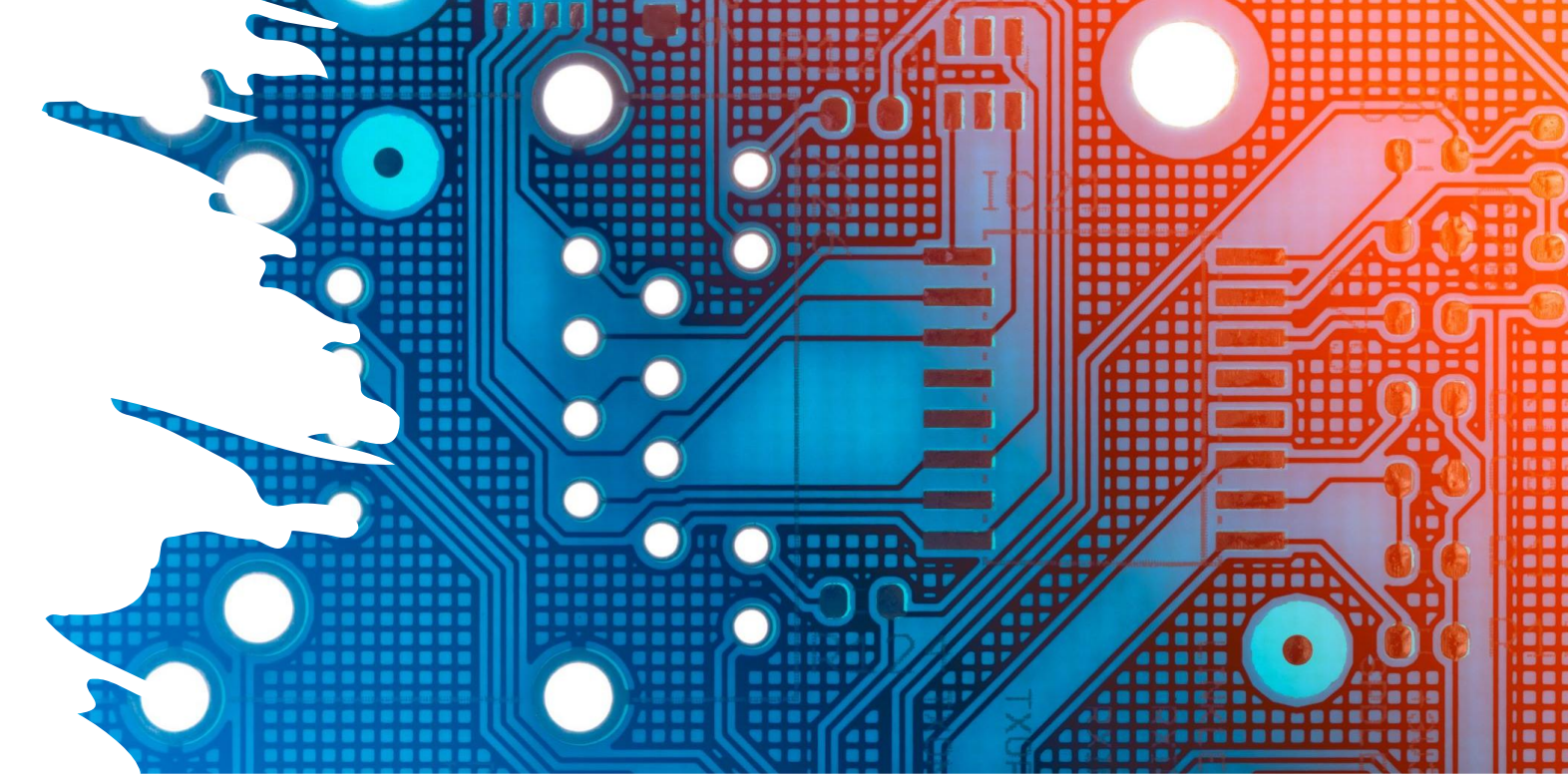
Architecture Spectrum

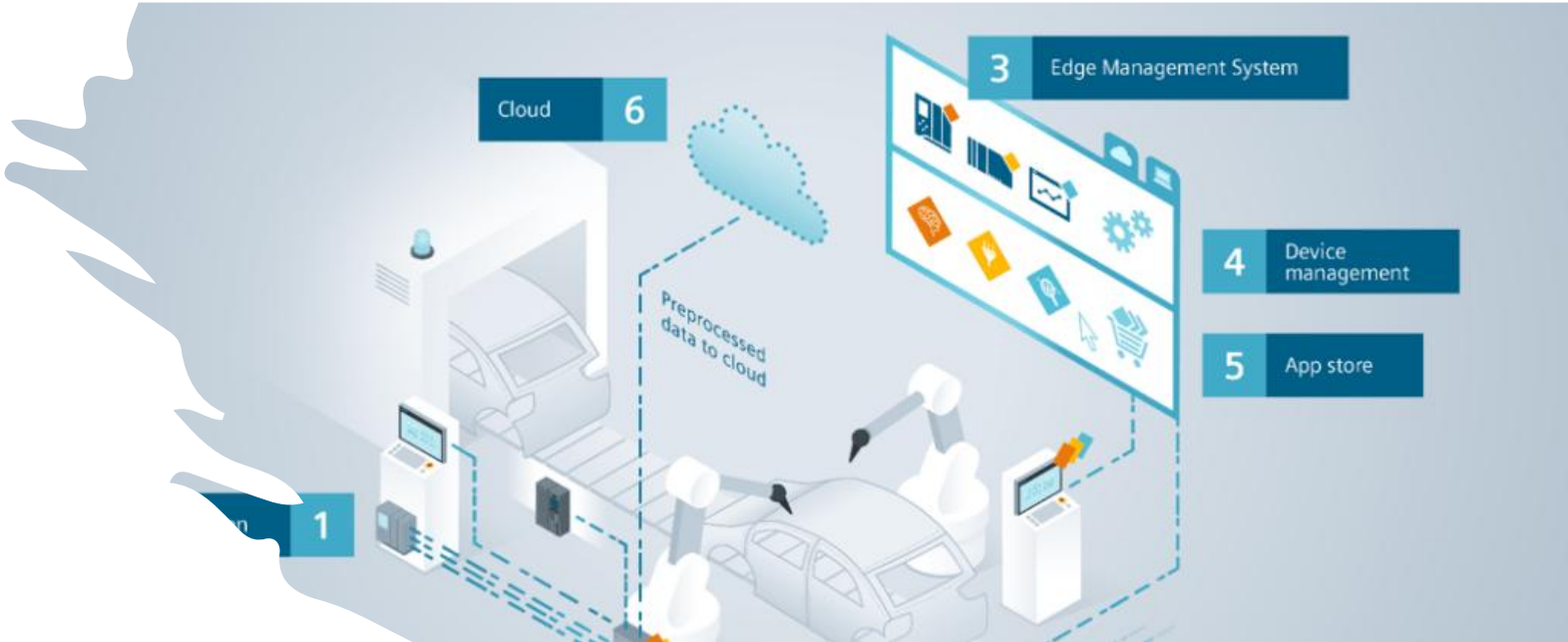
Edge → Federated → HPC → Kubernetes Cloud

Hybrid systems dominate in practice

Edge Computing

- Sensor → Local Compute → Decision → Actuation
- Ultra-low latency
- Must tolerate disconnection
- Power and memory constrained

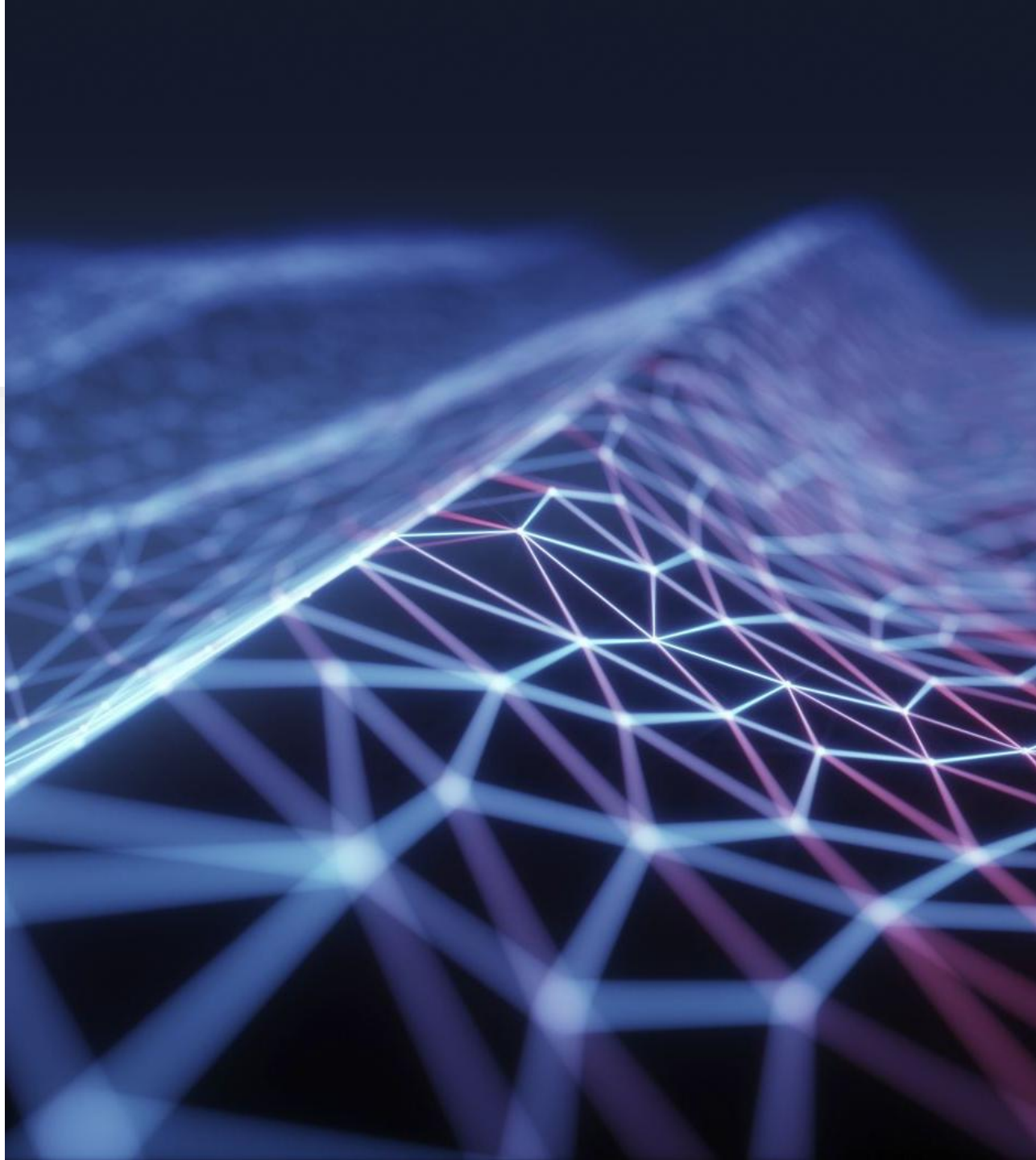




TECHLABS
POLYSYNC
cradlepoint

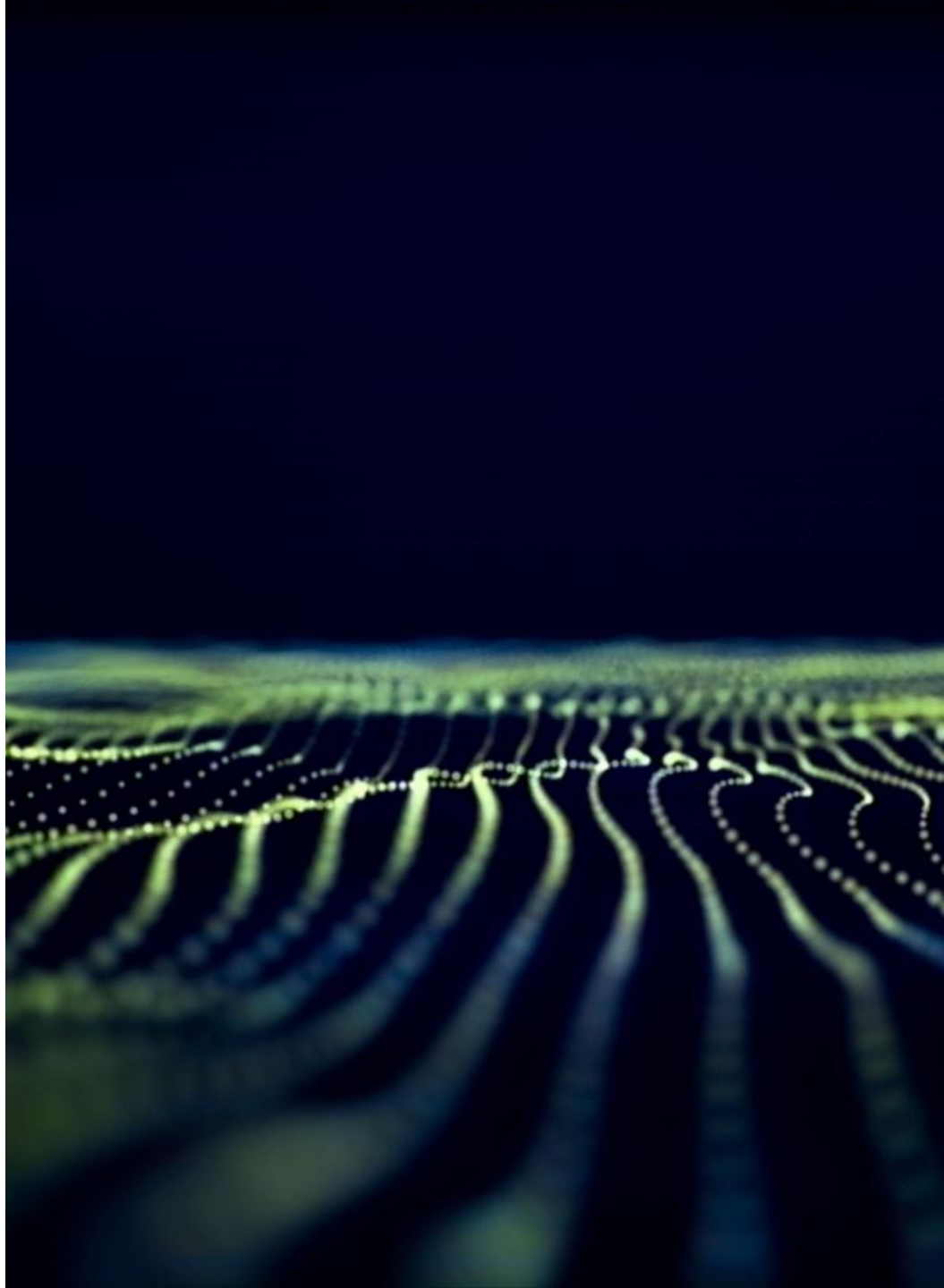
Human-Rich Robotics Example

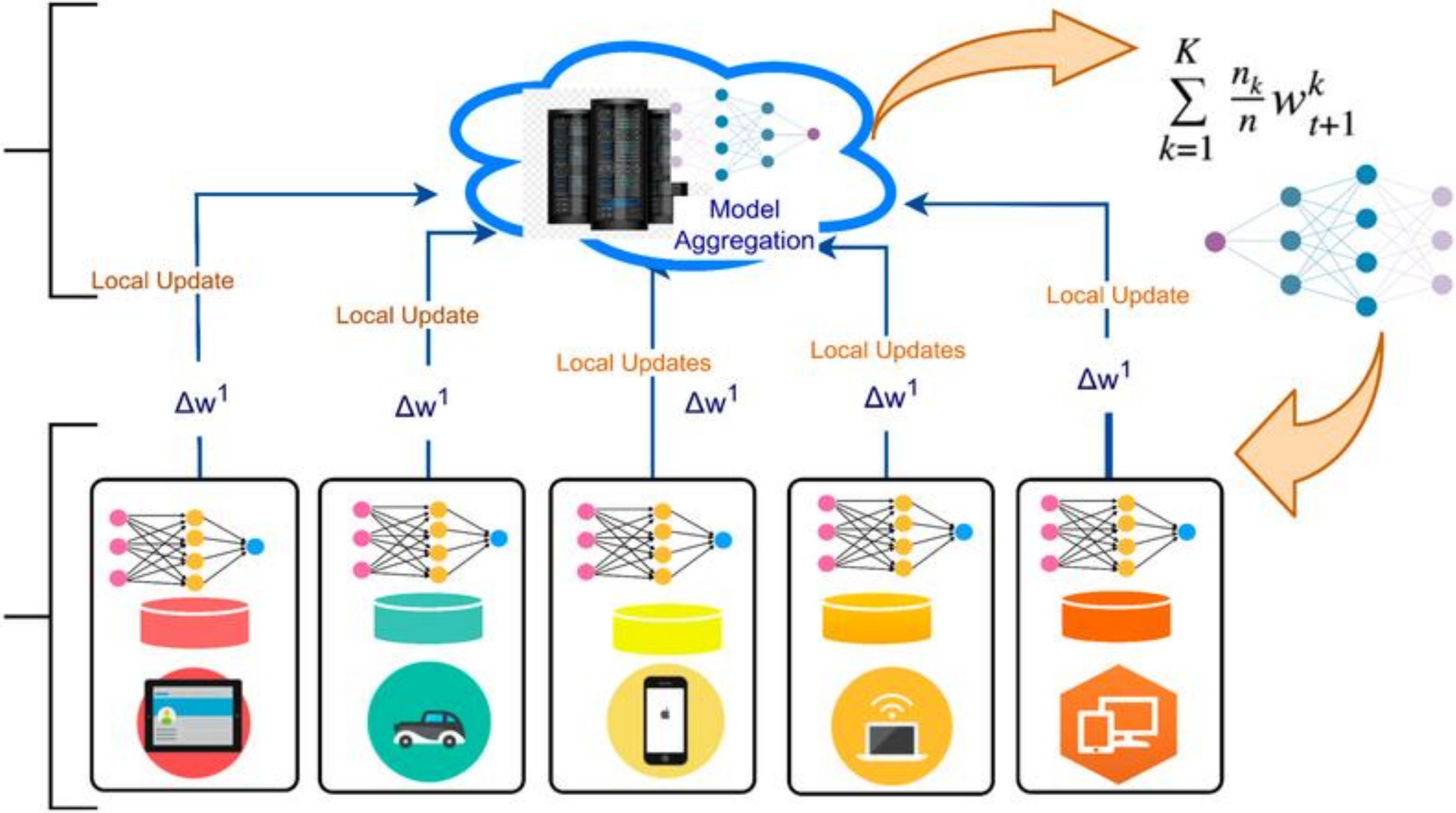
- Dynamic obstacle updates
- Probabilistic occupancy grids
- Receding-horizon replanning
- Continuous sensor fusion



Federated Systems

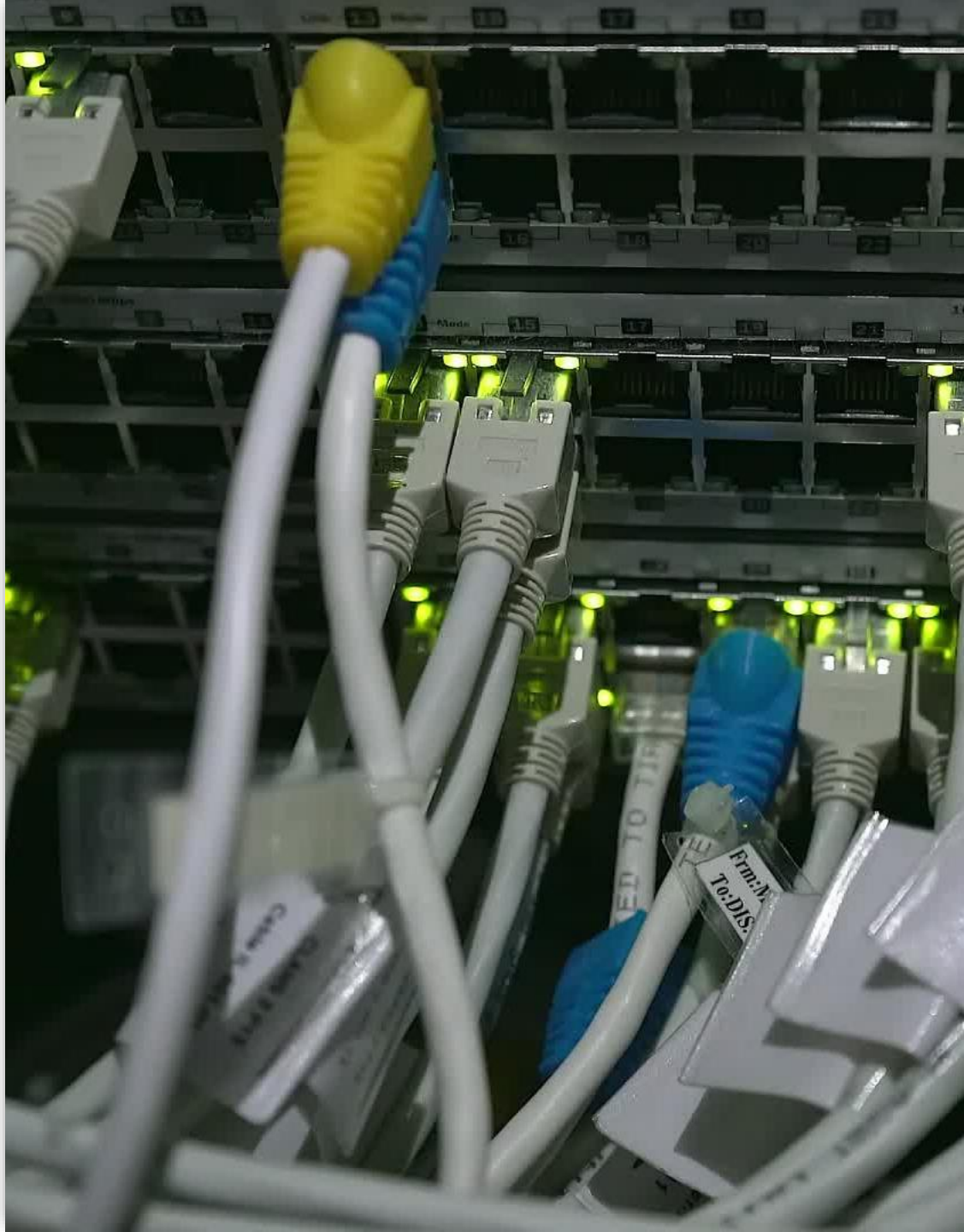
- Local training
- Gradient aggregation
- Privacy preservation
- Non-IID challenges




$$\sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^{k}$$
Model Aggregation
Local Update
Local Update
Local Updates
Local Updates
Local Update
Δw^1
Δw^1
Δw^1
Δw^1
Δw^1

HPC Clusters

- Head node and scheduler
- GPU compute nodes
- Shared storage
- Ideal for training and simulation



SLURM vs Kubernetes

SLURM: BATCH HPC SCHEDULING

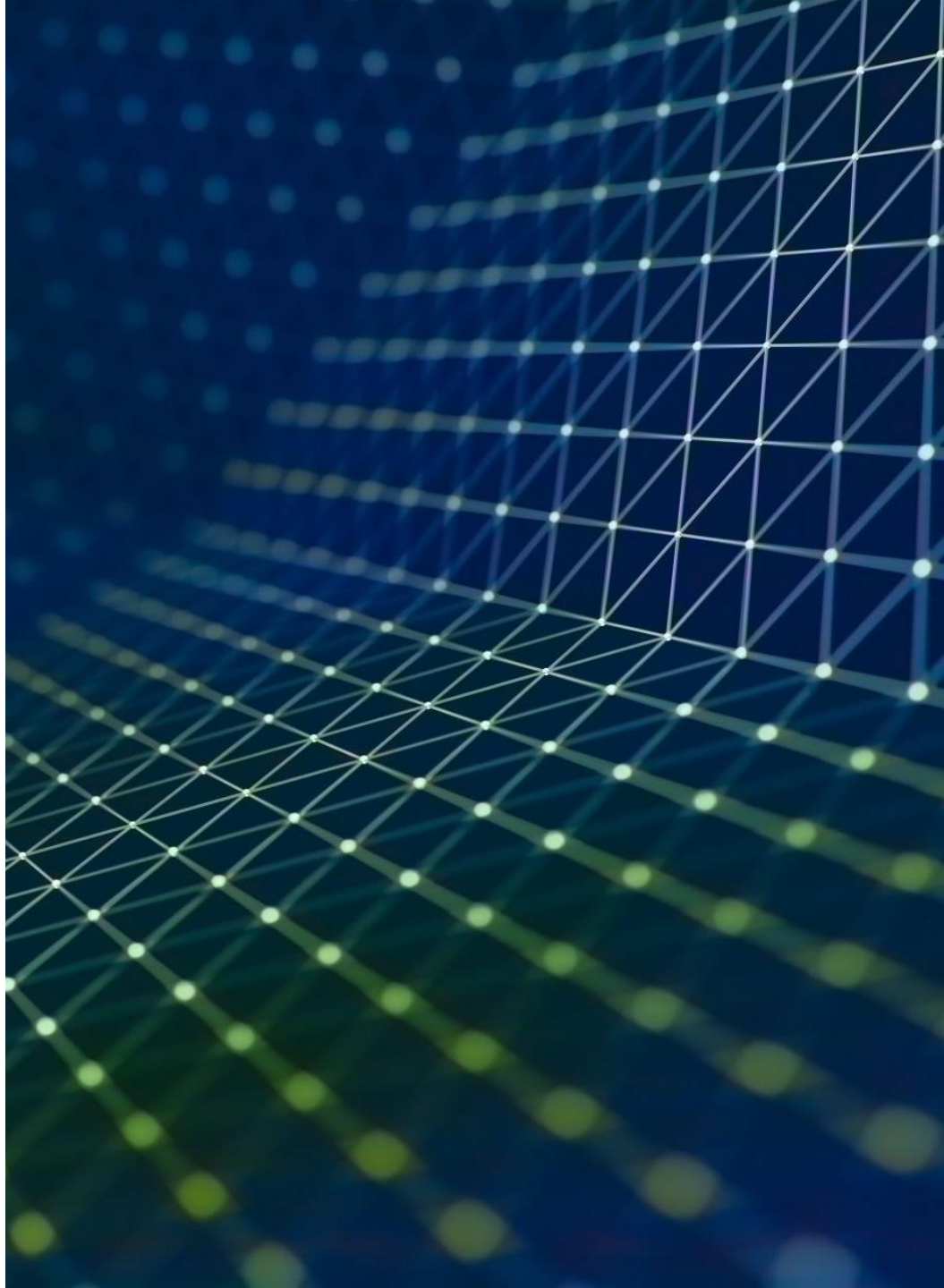
KUBERNETES: SERVICE ORCHESTRATION

BOTH GPU AWARE

DIFFERENT SCALING PHILOSOPHIES

Kubernetes Architecture

- Control plane
- Worker nodes
- Pods and services
- Auto scaling and self-healing



Design Axes in Distributed Systems

Where compute lives

How coordination occurs

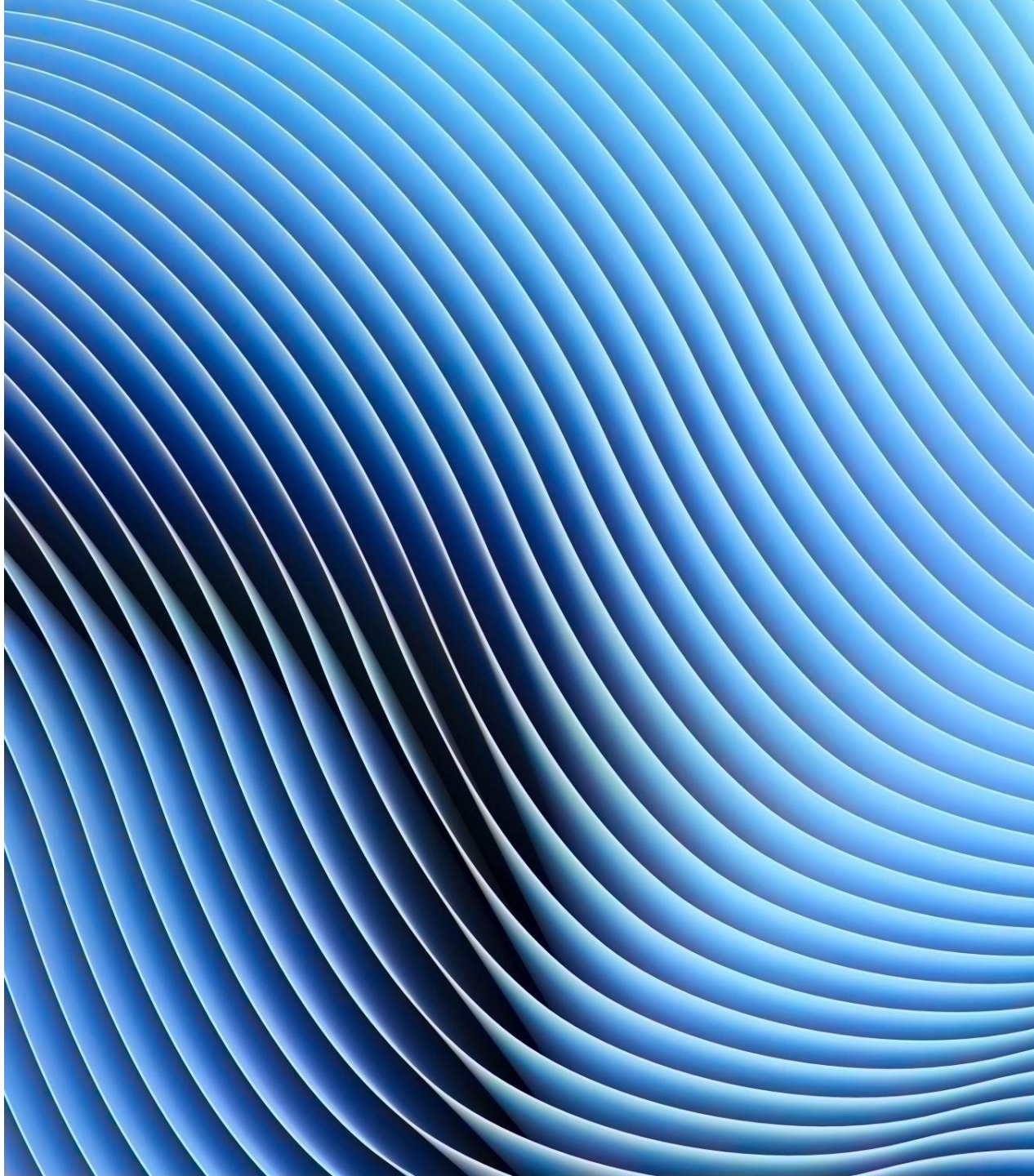
How failure is handled

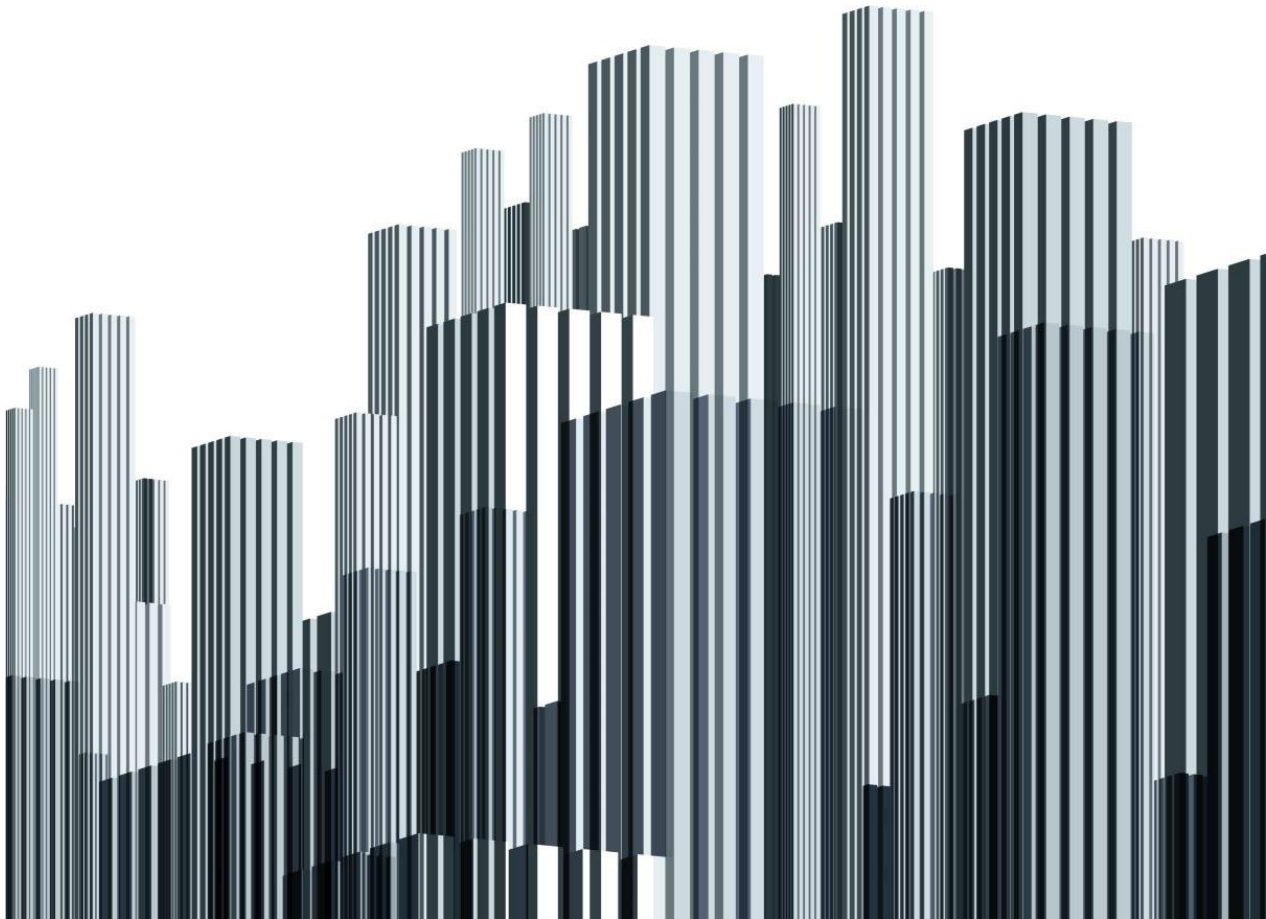
Real-World Example: Autonomous Fleet

- Edge inference onboard vehicles
- Federated updates across fleet
- HPC retraining
- Kubernetes dashboards and APIs

Failure Handling

- Edge failure
- Node failure
- Cluster failure
- Network partition



Closing Thoughts

- Hybrid architectures dominate
- Distributed intelligence is the future
- Design for scale and resilience